

Отчёт по лабораторной работе №8

Основы информационной безопасности

Татьяна Александровна Пономарева

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
5	Ответы на контрольные вопросы	9
6	Выводы	11
	Список литературы	12

Список иллюстраций

4.1	Создание и содержание файла lab08.py	8
4.2	Вывод файла lab08.py	8

Список таблиц

1 Цель работы

Освоить на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом.

2 Задание

Разработать приложение, позволяющее шифровать и дешифровать тексты P_1 и P_2 в режиме однократного гаммирования. Приложение должно уметь определять вид шифротекстов C_1 и C_2 обоих текстов P_1 и P_2 при известном ключе. Необходимо определить и выразить аналитический способ, при котором злоумышленник может прочесть оба текста, не зная ключа и не стремясь его определить.

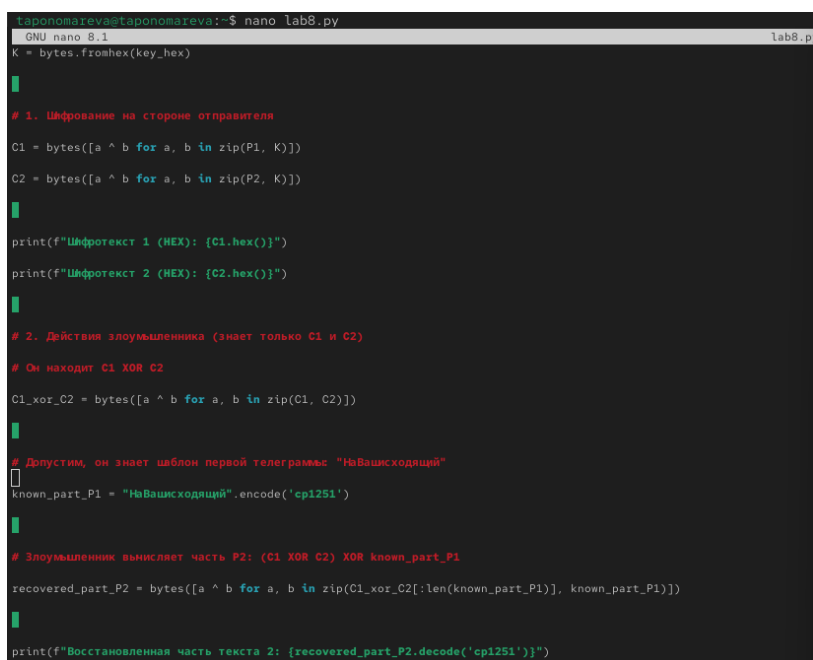
3 Теоретическое введение

Гаммирование – это наложение (снятие) на открытые (зашифрованные) данные криптографической гаммы, то есть последовательности элементов данных, вырабатываемых с помощью некоторого криптографического алгоритма, для получения зашифрованных (открытых) данных.

С точки зрения теории криптоанализа, метод шифрования случайной однократной равновероятной гаммой той же длины, что и открытый текст, является невскрываемым. Кроме того, даже раскрыв часть сообщения, дешифровщик не сможет хоть сколько-нибудь поправить положение – информация о вскрытом участке гаммы не дает информации об остальных ее частях. «Наложение» гаммы не что иное, как выполнение операции сложения по модулю 2 [1].

4 Выполнение лабораторной работы

Сначала создаем файл с кодом lab08.py (рис. 4.1).



```
taronomareva@taronomareva:~$ nano lab8.py
GNU nano 8.1 lab8.py
K = bytes.fromhex(key_hex)

# 1. Шифрование на стороне отправителя
C1 = bytes([a ^ b for a, b in zip(P1, K)])
C2 = bytes([a ^ b for a, b in zip(P2, K)])

print(f"Шфротекст 1 (HEX): {C1.hex()}")
print(f"Шфротекст 2 (HEX): {C2.hex()}")

# 2. Действия злоумышленника (знает только C1 и C2)
# Он находит C1 XOR C2
C1_xor_C2 = bytes([a ^ b for a, b in zip(C1, C2)])

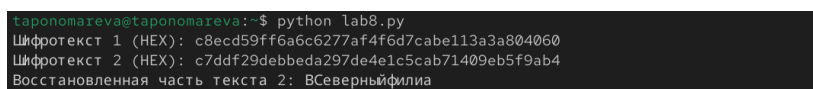
# Допустим, он знает шаблон первой телеграммы: "НаВашисходящий"
known_part_P1 = "НаВашисходящий".encode('cp1251')

# Злоумышленник вычисляет часть P2: (C1 XOR C2) XOR known_part_P1
recovered_part_P2 = bytes([a ^ b for a, b in zip(C1_xor_C2[:len(known_part_P1)], known_part_P1)])

print(f"Восстановленная часть текста 2: {recovered_part_P2.decode('cp1251')}")
```

Рисунок 4.1: Создание и содержание файла lab08.py

Получаем содержание второго кода, зная содержание первого (рис. 4.2).



```
taronomareva@taronomareva:~$ python lab8.py
Шфротекст 1 (HEX): c8ecd59ff6a6c6277af4f6d7cabe113a3a804060
Шфротекст 2 (HEX): c7ddf29debbda297de4e1c5cab71409eb5f9ab4
Восстановленная часть текста 2: ВСеверныйфилия
```

Рисунок 4.2: Вывод файла lab08.py

5 Ответы на контрольные вопросы

1. Как, зная один из текстов ((P_1) или (P_2)), определить другой, не зная при этом ключа?

Если известен один открытый текст, например (P_1), и соответствующие шифртексты (C_1) и (C_2) (полученные с одним и тем же ключом (K)), то можно восстановить ключ как $(K = C_1 \oplus P_1)$. Затем, используя этот ключ, расшифровать второй шифртекст: $(P_2 = C_2 \oplus K = C_2 \oplus C_1 \oplus P_1)$. Таким образом, знание одного текста позволяет вычислить другой без прямого знания ключа.

2. Что будет при повторном использовании ключа при шифровании текста?

Повторное использование одного и того же ключа (или гаммы) для двух разных открытых текстов нарушает стойкость шифра. Противник может вычислить $(C_1 \oplus C_2 = P_1 \oplus P_2)$, что даёт информацию о взаимосвязи текстов. Если один из текстов известен (или частично угадан), второй полностью раскрывается. Кроме того, возможен статистический анализ и восстановление обоих текстов. Такой режим лишает шифр свойства совершенной секретности, присущего однократной гамме.

3. Как реализуется режим шифрования однократного гаммирования одним ключом двух открытых текстов?

Для реализации генерируется случайный ключ (гамма) (K) длиной, равной длине открытых текстов. Затем вычисляются два шифртекста:

$$(C_1 = P_1 \oplus K),$$

$$(C_2 = P_2 \oplus K).$$

Один и тот же ключ (K) используется для обоих сообщений. Этот режим называют «двукратной гаммой» или «двухразовым блокнотом» (two-time pad).

4. Перечислите недостатки шифрования одним ключом двух открытых текстов.

- Потеря свойства совершенной секретности (однократная гамма требует одноразового использования ключа).
- При известном одном открытом тексте второй полностью раскрывается.
- Вычисление $(C_1 \oplus C_2 = P_1 \oplus P_2)$ даёт злоумышленнику возможность статистического анализа, частотного декодирования и восстановления текстов даже без знания исходных данных.
- Уязвимость к атакам на основе известного или подобранного открытого текста.
- Неприменимость для систем, требующих гарантированной стойкости.

5. Перечислите преимущества шифрования одним ключом двух открытых текстов.

- Экономия ключевого материала – требуется сгенерировать и распределить только один ключ для двух сообщений.
- Снижение вычислительной нагрузки (один процесс генерации гаммы вместо двух).
- В некоторых узкоспециализированных сценариях (например, при крайнем дефиците случайных последовательностей или для маскировки факта передачи двух сообщений) может быть полезно, но с пониманием потери стойкости.
- Простота реализации в системах, где допустимы указанные выше недостатки.

6 Выводы

В ходе проведения лабораторной работы было освоено на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом.

Список литературы

1. StudFiles. Режим шифрования однократного гаммирования: Программные средства защиты информации [Электронный ресурс]. НГТУ (Новосибирский государственный технический университет). URL: <https://studfile.net/preview/11550163/page:15/> (дата обращения: 20.04.2026).